



FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

BACHELOR OF COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH
HONORS

CSE3433 SOFTWARE ARCHITECTURE
SEMESTER II 2025/2026

GROUP 7 (K2)

Cloud-Native Learning Management System (LMS)

(Cloud-Native / Layered Architecture)

Prepared by:

NAME	MATRIC NUMBER
NUR ALIYA FARHANI BINTI MOHD NASRI	S75663
NUR ALIA FARISYA BINTI MOHD KHAIRY	S75892
ZIHAN FARISYA BINTI MOHD ZAINI	S75907

PREPARED FOR:
DR. MOHAMAD NOR HASSAN

SUBMISSION DATE:
19th JUNE 2026

Table of Contents

1.0 Introduction.....	2
2.0 Problem Description	3
3.0 System Requirements.....	5
4.0 Architecture Design	7
5.0 Architecture Diagrams	9
5.1 System Context Diagram	10
5.2 Component/Service Architecture Diagram	10
5.3 Interaction Diagram.....	10
5.4 Deployment Diagram	11
6.0 Technology Stack.....	11
7.0 Prototype Implementation.....	12
8.0 Architecture Evaluation	20
9.0 Team Contributions	22
10.0 Conclusion and Reflection.....	23

1.0 Introduction

The Cloud-Native Learning Management System (LMS) is a web-based application designed to support online learning activities such as user authentication and course access. The system is developed using a layered architecture approach and deployed on cloud infrastructure, with Aiven.io providing a managed MySQL database service.

The system enables three main types of users, which are learners, educators, and administrators, to interact with the LMS through a web interface. Students can register, log in, and access course information, while educators and administrators manage courses and monitor system usage.

The architecture of the LMS is divided into three main layers. The Presentation Layer provides the user interface, allowing users to perform actions such as login, registration, and accessing the course dashboard. The Application Layer handles the core business logic, including user authentication, session management, and course retrieval based on user roles. The Data Access Layer is responsible for managing communication with the cloud database, performing operations such as storing user credentials, retrieving course data, and maintaining session records.

The system uses Aiven.io, a cloud-managed database service, to ensure high availability, scalability, and data reliability. By leveraging Aiven's distributed cloud infrastructure, the LMS can handle multiple user requests efficiently while maintaining data consistency and security.

Overall, the Cloud-Native LMS provides a scalable, secure, and maintainable platform for delivering educational services, aligning with modern cloud computing and software architecture principles.

2.0 Problem Description

In modern higher education institutions, localized web portals and traditional monolithic Learning Management Systems (LMS) face critical operational boundaries. The core limitations observed in existing traditional university portal architectures include:

1. Infrastructure Bottlenecks and Server Degradation

Traditional university portals rely heavily on premise, localized hardware infrastructure. During high-concurrency peak academic periods, specifically during course registration weeks, examination releases, or final assignment submission deadlines, these localized application servers experience severe performance degradation. The sudden surge in concurrent student traffic creates massive computational bottlenecks, frequently leading to high latency, connection timeouts and total system crashes.

2. Lack of Dynamic Session and State Management

Many legacy student systems struggle with seamless state tracking across multiple web layers. When server loads spike, localized session boundaries often fail or expire prematurely. These forces authenticated students out of the system mid-transaction, leading to incomplete form submissions, data corruption during profile creation and an inconsistent user experience where user identity details fail to load dynamically across dashboard views.

3. Data Vulnerability and Maintenance Overhead

Maintaining localized database installations requires immense manual database administration (DBA) overhead, including manual backups, hardware patching and continuous performance tuning. If the physical local server suffers a hardware failure, the institution risks catastrophic data loss. Furthermore, localized relational databases lack built-in high availability and automated disaster recovery, making it difficult to scale storage dynamically as student enrollments increase.

4. Tight Structural Coupling (Monolithic Architecture)

Existing portals often combine user interface components, backend routing logic and database queries into tightly coupled files. This lack of architectural modularity means

that a minor error or bug within the database configuration can compromise the entire web application. Modifying the user interface layout requires modifying the core system backend, which drastically increases system downtime and complicates long-term maintenance by development teams.

3.0 System Requirements

Functional Requirements

The learning management system prototype focuses on two core modules that demonstrate the full architectural flow from the user interface to the Aiven.io cloud database.

Module 1: User Authentication

- FR-01: User Registration

The system shall allow new users to register by providing a username, email and password, which are then securely stored in the Aiven.io, MYSQL database.

- FR-02: Secure Login Authentication

The system shall verify user credentials against the stored records in the cloud database to grant access to the system.

- FR-03: Session Initialization

Upon successful login, the system shall create a session object to store user identity and maintain the login state during navigation.

- FR-04: Secure Logout

The system shall provide a logout function that terminates the user's session and redirects the user back to the login page.

Module 2: Course Dashboard

- FR-05: Authenticated Dashboard Access

The system shall ensure that only users with an active session can access the Course Dashboard module.

- FR-06: Dynamic Course Retrieval

The system shall fetch and display a list of available courses from the Aiven.io database based on the user's profile or enrollment status.

- FR-07: Course Detail View

The system shall allow users to select a specific course to view detailed information, such as course objectives and descriptions.

- FR-08: Session-Data Integration

The system shall display personalized information on the dashboard by retrieving data from the active session created in Module 1.

Non-Functional Requirements

- **Scalability:** The system must support independent scaling for each tier, allowing the Business Layer to handle increased traffic during peak periods, such as assignment submissions.
- **Availability:** By utilizing Aiven.io, the system aims for high availability through automated self-healing capabilities and distributed cloud nodes to minimize downtime.
- **Reliability:** The Data Access Layer must ensure data integrity and consistency through automated backups and secure SQL transactions within the Aiven cloud environment.
- **Security:** Access control must be strictly enforced, ensuring that only authenticated users with active sessions can access the dashboard and sensitive course materials.
- **Portability:** The architecture shall remain cloud-independent, allowing the LMS to be deployed or migrated across different cloud service providers without significant changes to the core logic.

4.0 Architecture Design

The Cloud-Native Learning Management System (LMS) is a web-based application developed using a layered architecture approach and deployed on cloud infrastructure. The system architecture is divided into three primary layers to ensure a separation of concerns and efficient delivery of educational services. The Presentation Layer serves as the user interface, providing web-based forms for user authentication and a dynamic course dashboard for learners, educators and administrators. Beneath this, the Application Layer handles core logic, including credential verification, session management and course retrieval based on user roles. The final layer, the Data Access Layer, manages communication with the cloud database using a Data Access Object (DAO) and a Cloud Connectivity Provider to perform secure operations such as storing credentials and maintaining session records.

By integrating these layers with cloud-native principles, the system leverages an Aiven.io managed MySQL service to ensure high availability, scalability and data reliability. This architectural style allows the Business Layer to scale independently to handle traffic spikes, while Aiven's distributed cloud infrastructure ensures multiple user requests are handled with consistent data security. Additionally, the architecture is designed for portability, remaining cloud-independent so the LMS can be migrated across different providers without requiring significant changes to the core logic. Overall, this layered, cloud-native approach provides a secure, scalable and maintainable platform that aligns with modern software architecture principles.

1. Presentation Tier Components

- **User Authentication Interface:** A web-based form that captures user credentials (email/password) for registration and login processes.
- **Course Dashboard UI:** A dynamic interface that displays personalized course lists, descriptions and user profile information after a successful session is established.

2. Business Tier Services

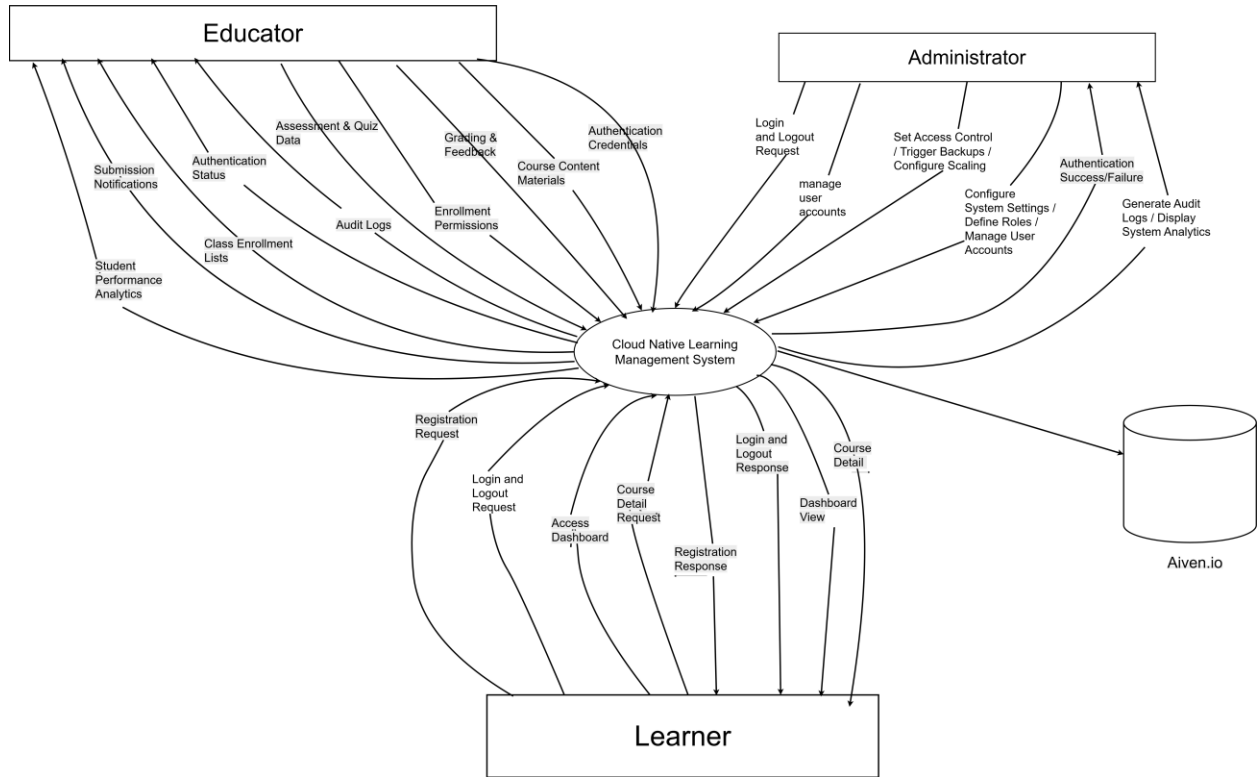
- **Authentication Service (Authentication Manager):** Responsible for validating user input, verifying credentials against the database and managing secure session objects.
- **Course Management Service:** Manages the logic for retrieving available courses from the cloud database based on the specific user's enrollment status.
- **Session Controller:** A security component that monitors active sessions to ensure only authenticated users can access restricted modules like the Course Dashboard.

3. Data Tier and Infrastructure Services

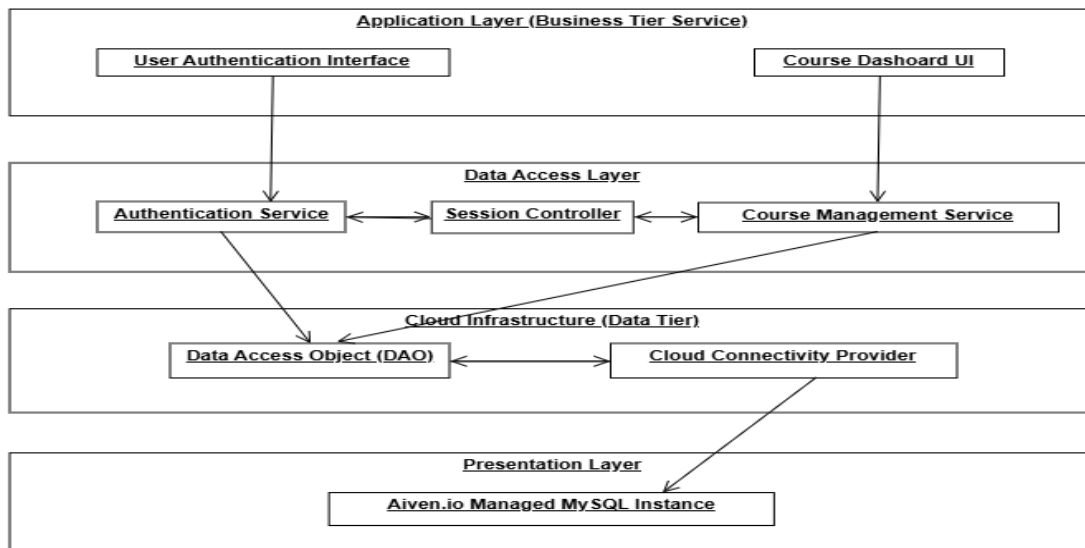
- **Data Access Object (DAO):** A structural component that provides an interface for the Business Tier to execute SQL queries (CRUD operations) without directly interacting with the database engine.
- **Aiven.io Managed MySQL Instance:** A cloud-native database service that provides high availability, automated backups and self-healing storage for all LMS data.
- **Cloud Connectivity Provider:** Ensures a secure connection between the application server and the Aiven.io cloud environment.

5.0 Architecture Diagrams

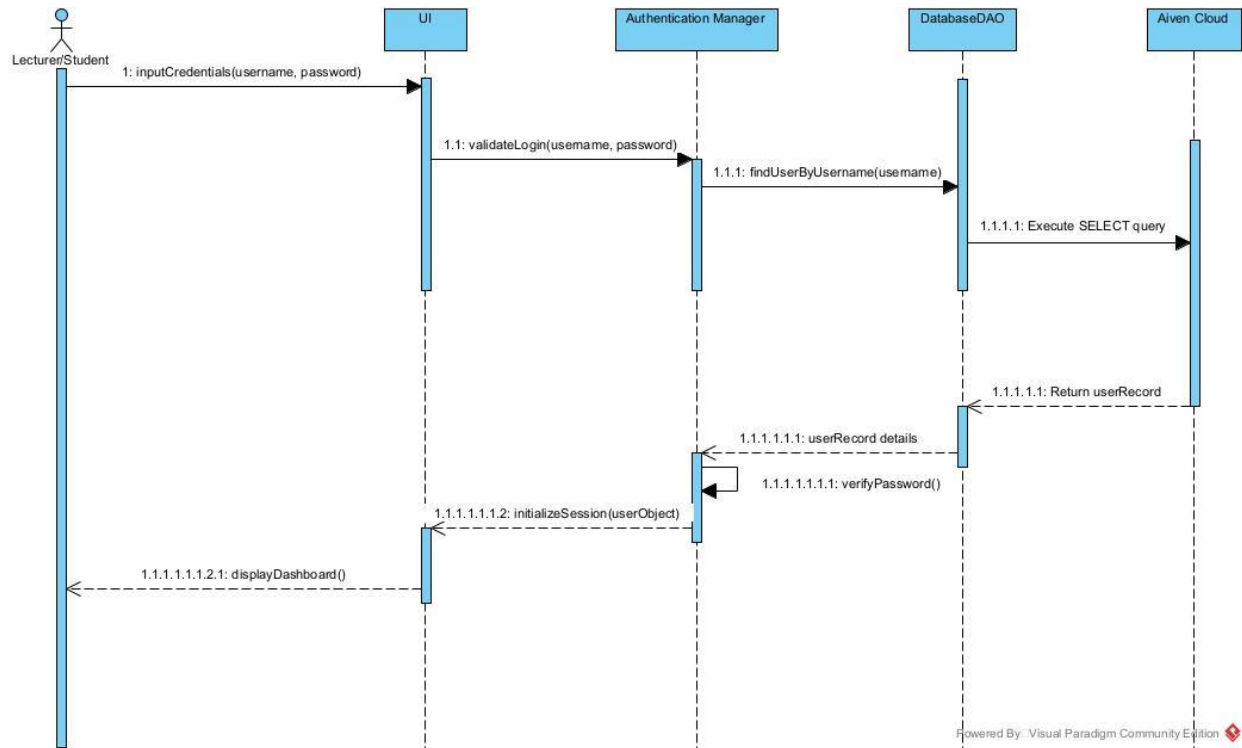
5.1 System Context Diagram



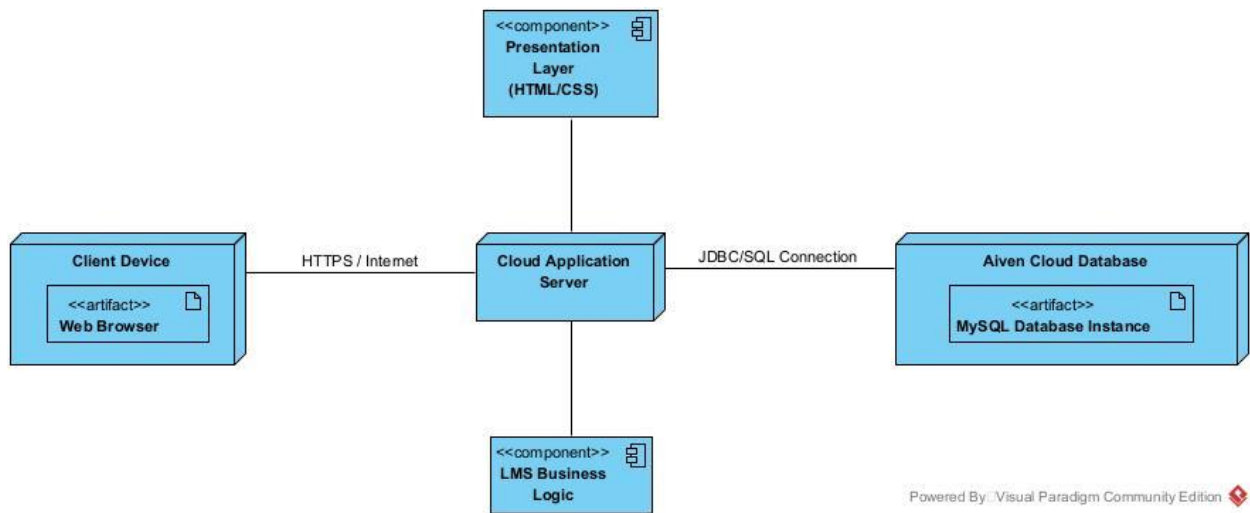
5.2 Component/Service Architecture Diagram



5.3 Interaction Diagram



5.4 Deployment Diagram



6.0 Technology Stack

To realize the three-tier layered architecture and achieve a resilient, cloud-native learning management system, the following technology stack has been selected to ensure independent scalability and high availability.

System Layer	Technology Chosen	Architectural Justification & Mapping
Presentation Layer	HTML5, CSS3 & JavaScript	- Responsible for building the static and interactive user interface (UI) components for register.jsp and login.jsp. - Utilizes HTML5 native input constraints for basic client-side form validations before sending requests to the backend.
Business Logic Layer	Servlets & JSP Scriptlets	- Handles core system logic, request parameter parsing (request.getParameter), and dynamic server-side evaluations. - Manages secure credential verification via AuthServlet.java and handles user routing transitions.
Data Access Layer	MySQL hosted on Aiven.io	- Provides a cloud-managed relational database environment to securely store user credentials via UserDAO.java. - Fulfill automated self-healing and data consistency.

User Interface and System Walkthrough

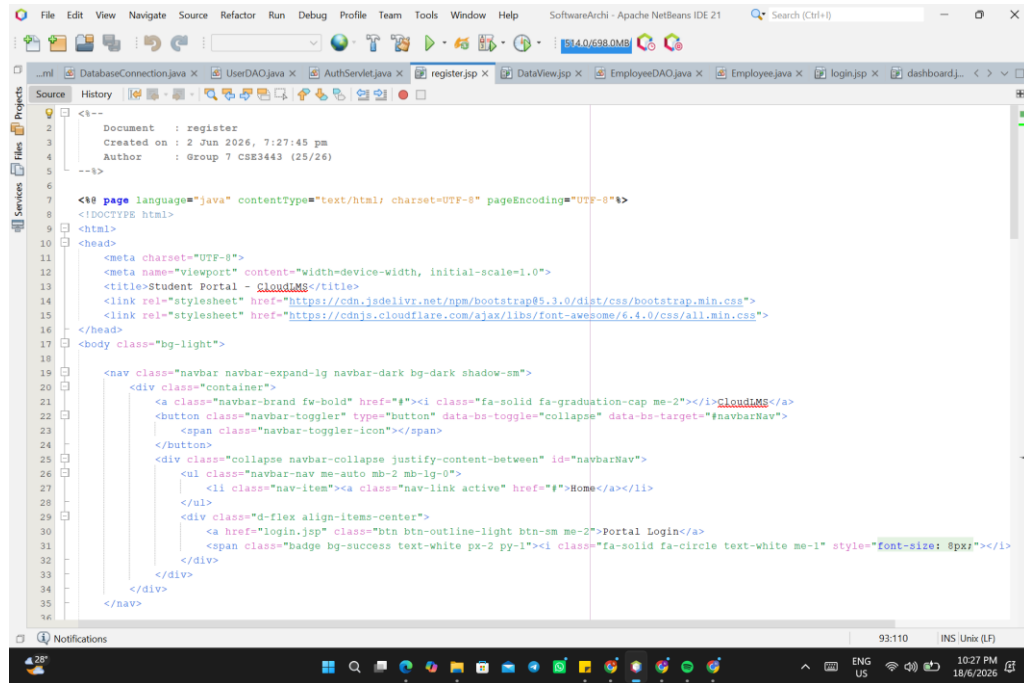


Figure 1: Top Navigation Bar and Metadata Configuration for CloudLMS Registration

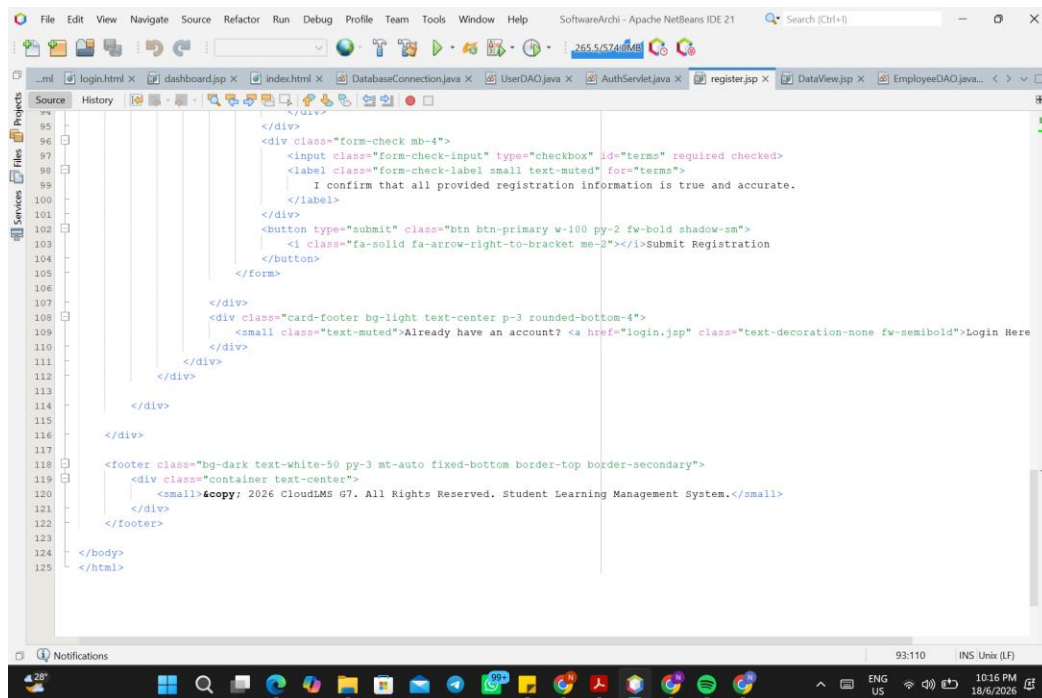


Figure 2: Server-side Java Scriptlet Logic Evaluating Request Parameters

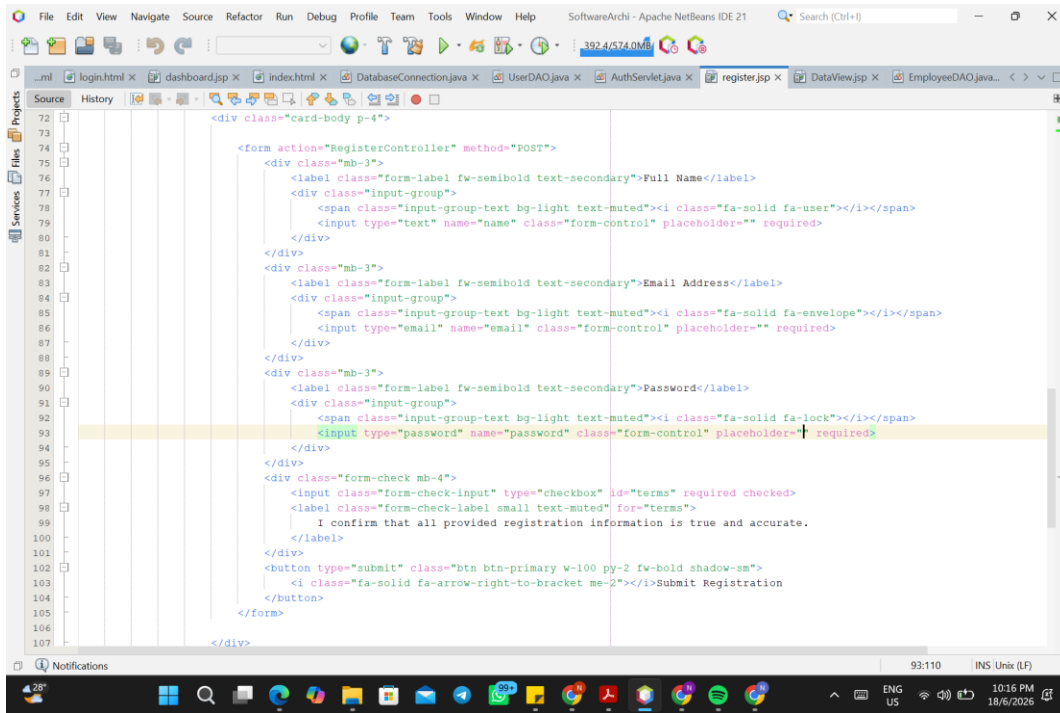


Figure 3: Data-dense Input Group Elements and Form Actions Mapping

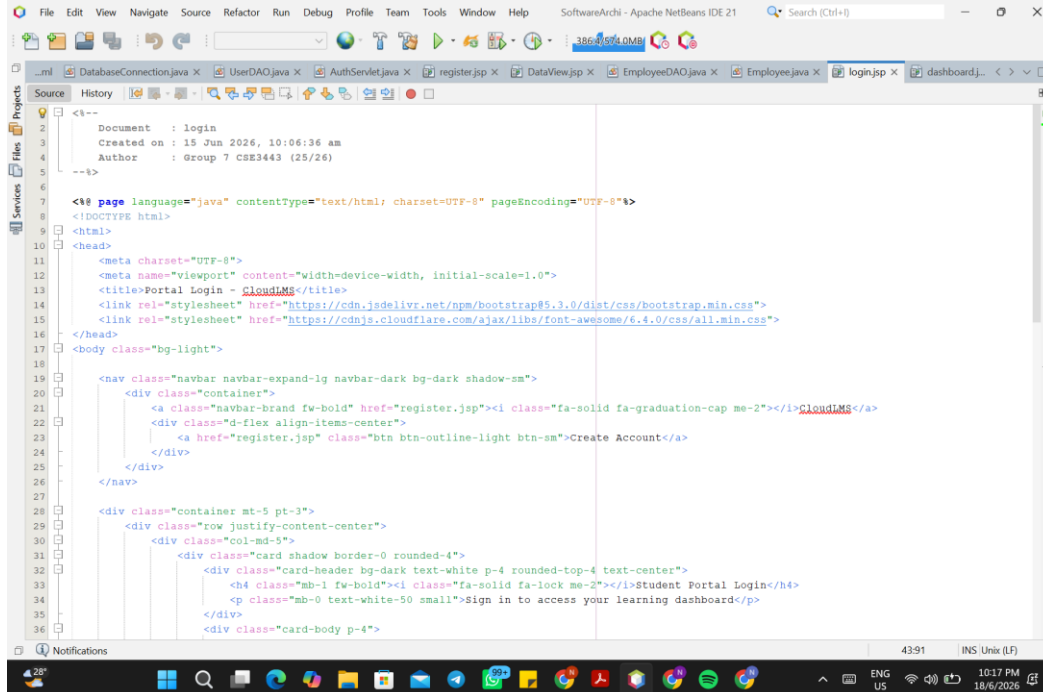


Figure 4: Upper Framework Setup and Structural Navigation for the Login Portal

```
37 <form action="dashboard.jsp" method="GET">
38 <div class="mb-3">
39 <label class="form-label fw-semibold text-secondary">Email Address</label>
40 <div class="input-group">
41 <span class="input-group-text bg-light text-muted"><i class="fa-solid fa-envelope"></i></span>
42 <input type="email" class="form-control" placeholder="" required>
43 </div>
44 </div>
45 <div class="mb-4">
46 <label class="form-label fw-semibold text-secondary">Password</label>
47 <div class="input-group">
48 <span class="input-group-text bg-light text-muted"><i class="fa-solid fa-lock"></i></span>
49 <input type="password" class="form-control" placeholder="" required>
50 </div>
51 <div class="text-end mt-2">
52 <a href="#" class="text-decoration-none small text-muted">Forgot Password?</a>
53 </div>
54 <div>
55 <button type="submit" class="btn btn-dark w-100 py-2 fw-bold shadow-sm">
56 <i class="fa-solid fa-arrow-right-to-bracket me-2"></i>Log In
57 </button>
58 </div>
59 </form>
60
61 <div>
62 <div class="card-footer bg-light text-center p-3 rounded-bottom-4">
63 <small class="text-muted">New student? <a href="register.jsp" class="text-decoration-none fw-semibold">Register Here</a></small>
64 </div>
65 </div>
66 </div>
67 </div>
68
69 <div>
70 <div class="bg-dark text-white-50 py-3 mt-5 fixed-bottom border-top border-secondary">
71 <div class="container text-center">
72 <small>&copy; 2026 CloudIMS G7. All Rights Reserved.</small>
73 </div>
74 </div>
75 </div>
76 </body>
77 </html>
```

Figure 5: Input Groups for Student Credentials and Target Destination Endpoints

```
44 </div>
45 <div>
46 <div class="mb-4">
47 <label class="form-label fw-semibold text-secondary">Password</label>
48 <div class="input-group">
49 <span class="input-group-text bg-light text-muted"><i class="fa-solid fa-lock"></i></span>
50 <input type="password" class="form-control" placeholder="" required>
51 </div>
52 <div class="text-end mt-2">
53 <a href="#" class="text-decoration-none small text-muted">Forgot Password?</a>
54 </div>
55 <div>
56 <button type="submit" class="btn btn-dark w-100 py-2 fw-bold shadow-sm">
57 <i class="fa-solid fa-arrow-right-to-bracket me-2"></i>Log In
58 </button>
59 </div>
60 </form>
61
62 <div>
63 <div class="card-footer bg-light text-center p-3 rounded-bottom-4">
64 <small class="text-muted">New student? <a href="register.jsp" class="text-decoration-none fw-semibold">Register Here</a></small>
65 </div>
66 </div>
67 </div>
68
69 <div>
70 <div class="bg-dark text-white-50 py-3 mt-5 fixed-bottom border-top border-secondary">
71 <div class="container text-center">
72 <small>&copy; 2026 CloudIMS G7. All Rights Reserved.</small>
73 </div>
74 </div>
75 </div>
76 </body>
77 </html>
```

Figure 6: Lower Card Footer Links and System Copyright Boundaries

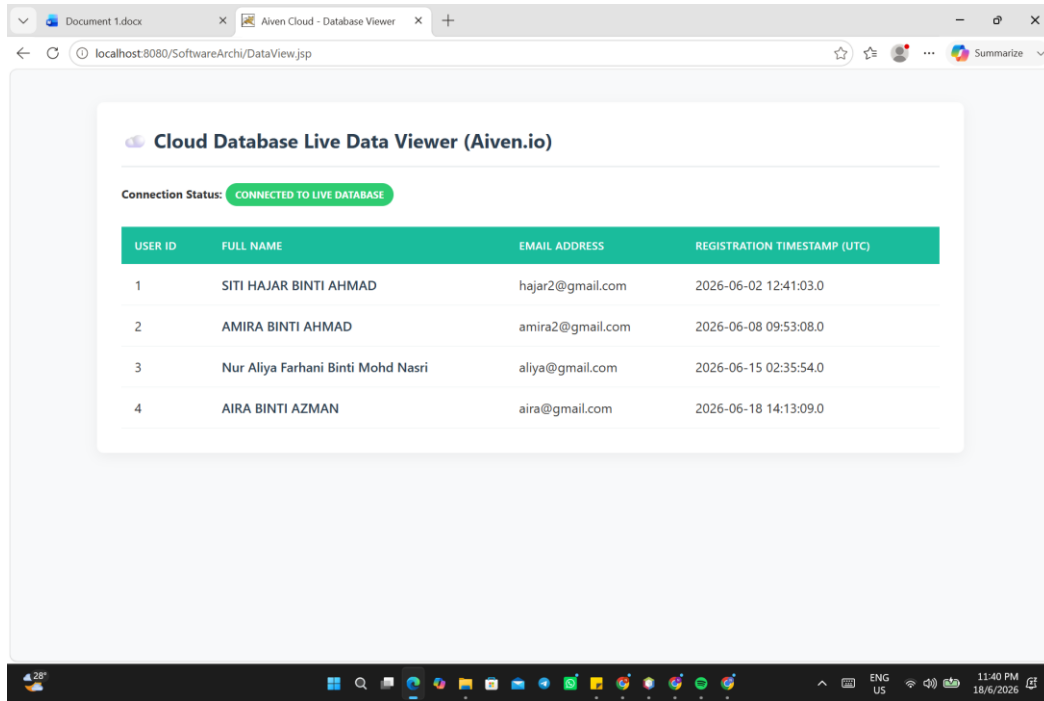


Figure 7: Graphical runtime viewport displaying live data fetched from the Aiven.io cluster

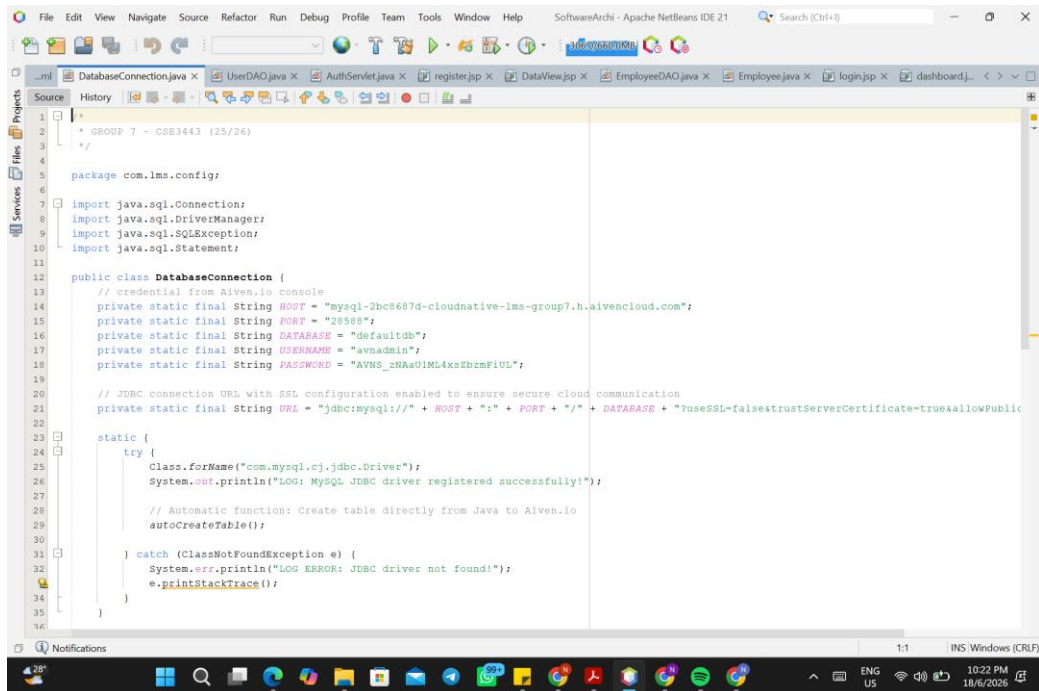


Figure 8: Class Definition and Baseline JDBC Driver Allocation Infrastructure

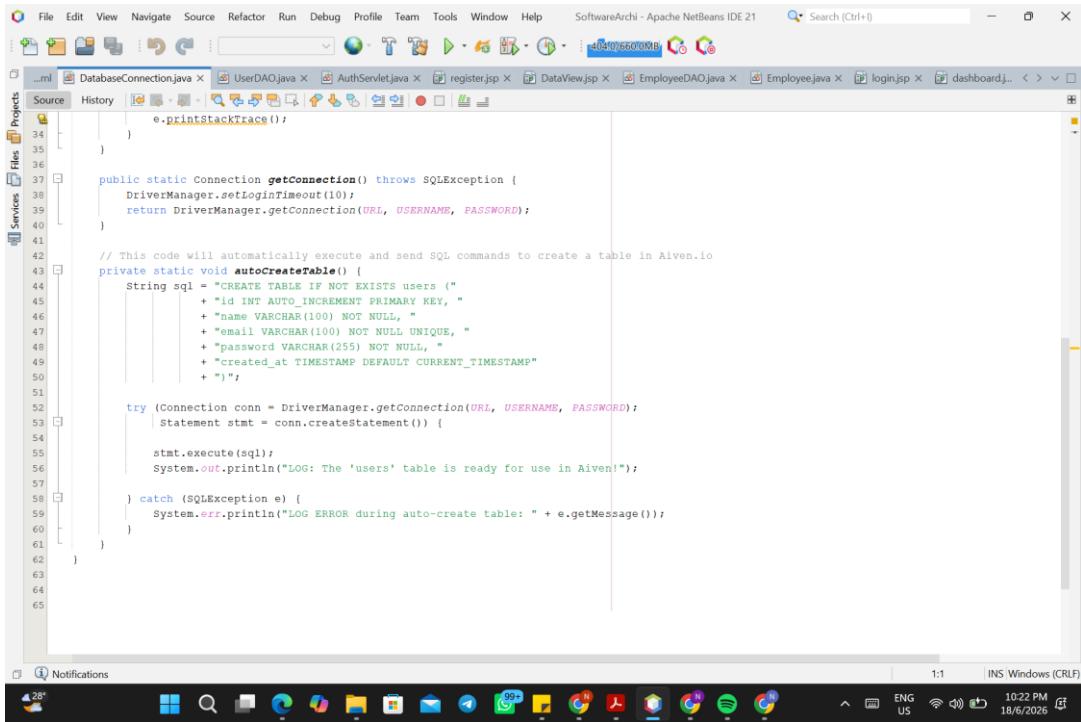


Figure 9: JDBC Link Establishment Mapping Directly to the Remote Cloud Engine

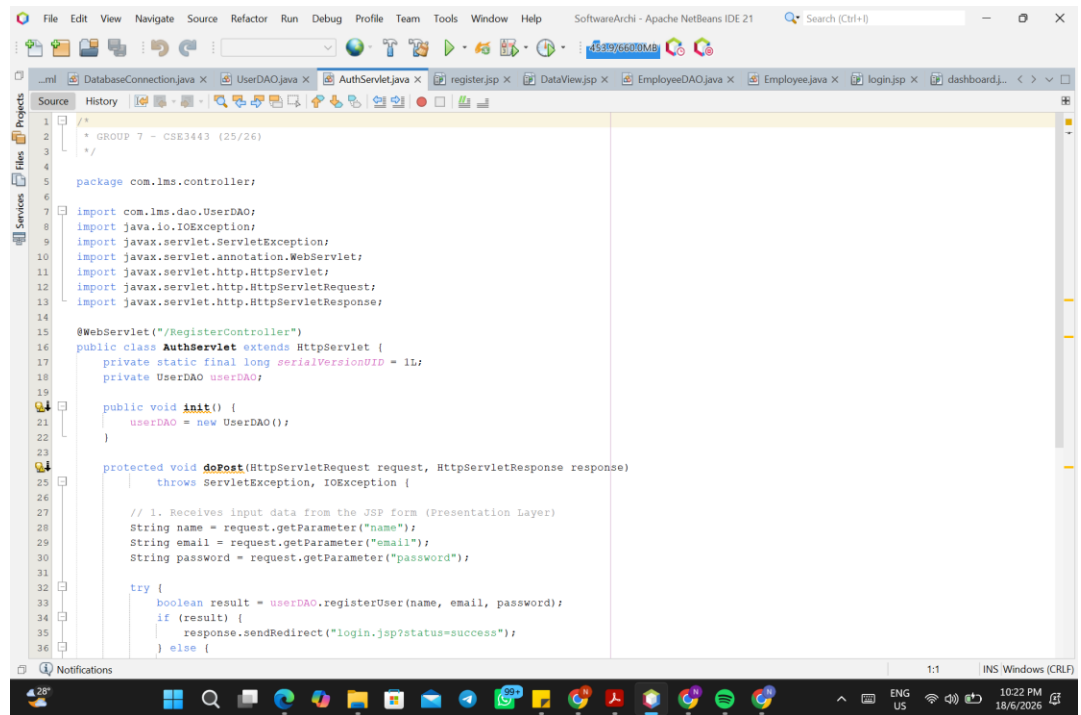


Figure 10: Servlet Mappings, Component Initialization and Request Parsing Configurations

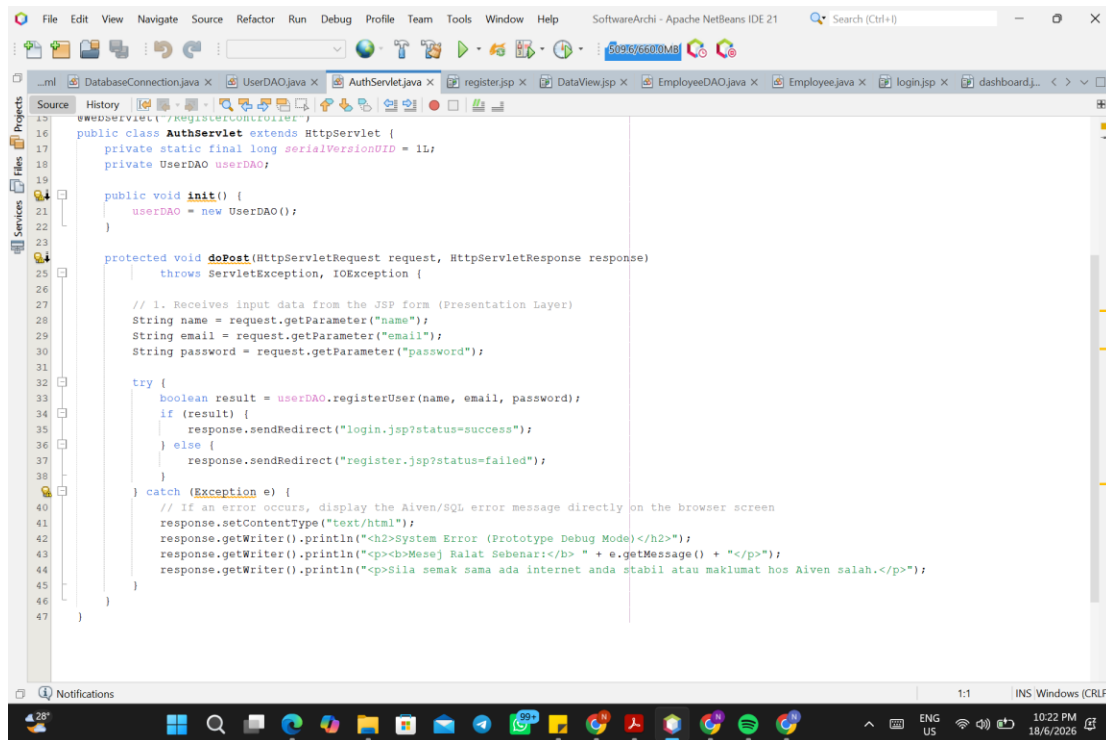


Figure 11: Conditional Response Redirection Rules and Localized Error Debugging Blocks

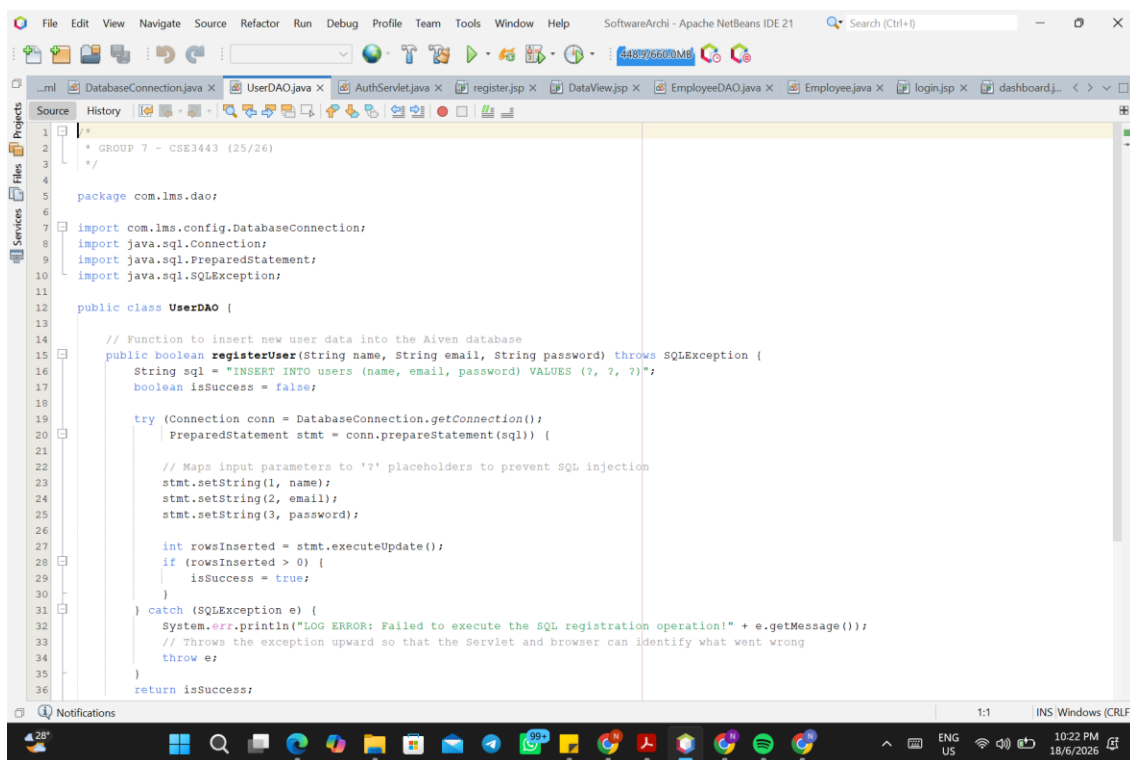


Figure 12: Data Access Object (DAO) Implementation with Parameterized SQL Injection Prevention

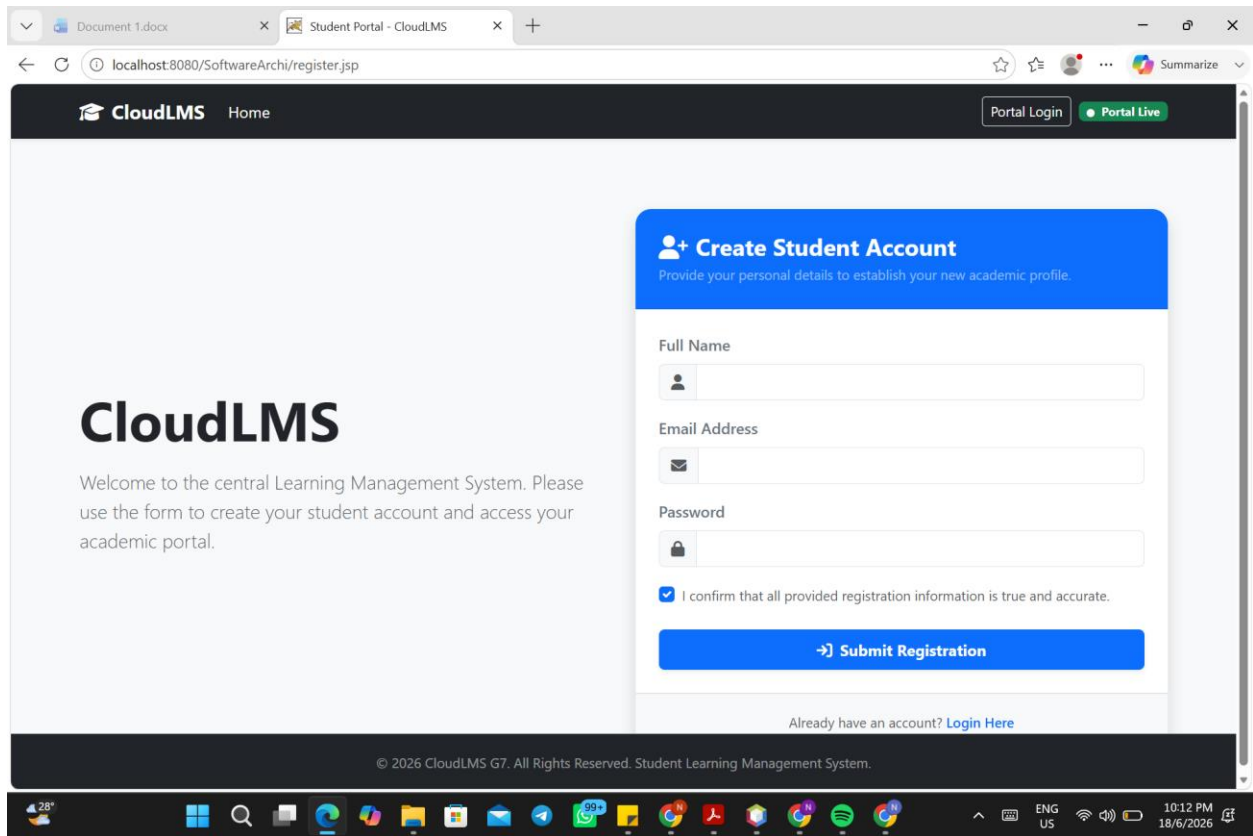


Figure 13: State Layout of the CloudLMS Student Registration Workspace

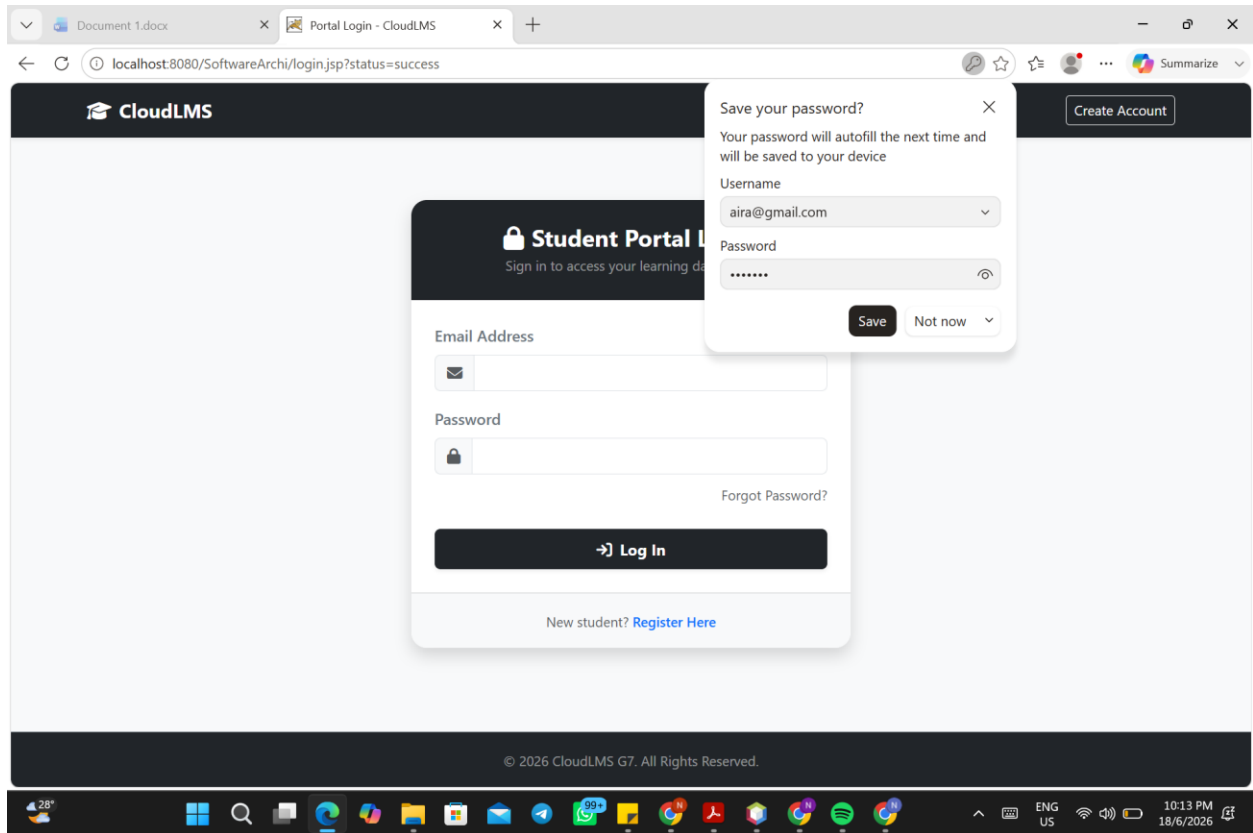


Figure 14: Redirection Landing Page Showing a Successful Database Status Parameter Loop

8.0 Architecture Evaluation

The layered architecture implemented in the Cloud-Native Learning Management System prototype demonstrates several strengths in meeting the functional and non-functional requirements. The system includes separate Presentation, Application, and Data Access Layers, ensuring that modifications to one layer do not impact the others. For example, changes to the login user interface can be made without altering the AuthServlet or UserDAO components. This separation significantly improves system maintainability and reduces the risk of development errors when code updates are performed.

Aiven.io provides a managed cloud database with advantages such as automated backups and self-healing infrastructure, which strengthen the Data Access Layer. Testing confirms that the DataView module can successfully retrieve data directly from the live database, demonstrating that the Data Access Layer operates independently of local server availability. This independence ensures that data persistence no longer relies on the local machine being online, thereby enhancing system reliability.

The implementation of the DAO pattern through the UserDAO component follows industry best practices. The use of parameterized queries via PreparedStatement effectively reduces the risk of SQL injection attacks, contributing to the overall security of user data.

In summary, the layered architecture chosen for this system successfully addresses critical issues related to data vulnerability and tight structural coupling. The integration of Aiven.io alongside the DAO pattern provides a secure and reliable approach to data storage and access while effectively preventing SQL injection vulnerabilities. A summary of this evaluation is shown in the table below.

Architectural Properties	Judgment	Description
Maintainability	High	Clear separation between Presentation, Application, and Data Access Layers allows modifications to be made to specific components without affecting others
Scalability	Moderate	The data store can scale easily through Aiven.io managed database, but the application tier cannot scale independently as it runs as a single WAR file on one Tomcat server
Availability	Moderate	High availability is provided for the database through Aiven.io, but the application server remains a single point of failure
Security	Moderate	SQL injection is prevented through parameterized queries using PreparedStatement, but error messages should be sanitized before being displayed to users
Portability	High	The Data Access Layer can be connected to another SQL database provider with minimal changes required

Modularity	High	Separation of concerns is achieved between Business Logic and Data Access components through the DAO pattern
------------	------	--

9.0 Team Contributions

Team Member	Academic Role	Key Architectural & Implementation Duties
Nur Alia Farisya Binti Mohd Khairy (S75892)	Architect & DevOps Lead	- Designed the multi-tier system architecture layers. - Drafted the multi-cloud delivery infrastructure and deployment diagrams. - Selected and justified the integrated framework technology stack.
Nur Aliya Farhani Binti Mohd Nasri (S75663)	Backend & Database Lead	- Implemented backend Java Servlet controllers to handle request processing. - Mapped baseline JDBC database connectivity configurations. - Wrote parameterized SQL transaction methods to prevent SQL injection. - Configured and initialized live relational schema setups on Aiven.io.
Zihan Farisya Binti Mohd Zaini (S75907)	Frontend & UI Developer	- Authored the web presentation framework views and structure (.jsp layouts). - Developed responsive to native HTML5 validation constraints for registration and login interface components.

		Handled the overall architectural and performance evaluation metrics of the system.
--	--	---

10.0 Conclusion and Reflection

The Cloud-Native LMS prototype demonstrates the realization of a strict Three-layer Architecture. The system moves from a legacy, tightly coupled architecture to a clearly modular platform, achieving a successful demarcation between layers and avoiding common architectural anti-patterns such as presentation layers directly interacting with the database. The division of the system into separate feature-centric modules provides logical partitions that prevent the LMS from becoming a monolith in disguise as individual components. And the fact that Aiven.io has opted for a cloud-managed relational database cluster is perfectly in line with the very nature of cloud-native objectives: providing high availability, real-time automated backups and quick self-recovery.